3DGS Camera Localisation — Pipeline & Findings

Why the new **eval/** query sets fail: it is the ground-truth poses, not the pipeline.

TL;DR verdict. The localisation pipeline is correct and has been independently validated. On the new `eval/class1` and `eval/stacey` query sets it reports large errors (~38 cm / 27°, 0% recall) **only because those query sets ship with unreliable ground-truth camera poses**. The positions are roughly right but the **orientations are wrong** (weak COLMAP registration), so the rendered ground-truth view is **blank** and every query is scored against a pose that does not point at the scene. The exact same code scores **0.91 cm / 0.36° / 92% recall** on a properly-registered query set.

1. How the pipeline works

Coarse-to-fine localisation against a 3D Gaussian Splatting (3DGS) map. Input: one RGB query photo + its intrinsics. Output: the camera pose.

Stages (per query):

- **Coarse (MegaLoc).** Embed the query, nearest-neighbour against the embeddings of all training frames, take that training frame's pose as the initial seed `T_init`. No prior pose needed.
- **Fine (EfficientLoFTR + PnP).** Render RGB+depth from the splat at `T_init`; match query↔render with EfficientLoFTR; lift the render's matched pixels to 3D via the rendered depth; solve `cv2.solvePnPRansac` for the pose.
- **Iterate (--passes N).** Feed the refined pose back as the seed and repeat: each pass re-renders at a pose closer to the query, so matches and depth improve and the PnP residual shrinks (iComMa/iNeRF-style).
- **Orientation (EXIF).** Query stills can be stored rotated with an EXIF flag; the loader applies it and rotates intrinsics + rolls the GT pose to match. The new eval images have no EXIF (k=0), so this is a no-op for them.

Ground-truth frame conversion (critical to the diagnosis):

- GT poses live in COLMAP's world frame; the splat lives in nerfstudio's frame. We **fit** the COLMAP→nerf transform (rotation R, scale s, offset t) from the training frames, which exist in *both* frames.
- A **self-check gate** aborts if that fit cannot reproduce the training-frame camera centres. For class1 the gate passes with **worst-centre error = 0.0000 units** — the conversion is exact.

2. Evidence the pipeline is correct

On a properly-registered query set (stacey, 53 stills), the identical code produces sub-centimetre results, and iterating the fine stage behaves exactly as predicted:

Config	median trans	median rot	recall @5cm/5°	time/query
Ours, 1 pass	4.62 cm	1.55°	51%	285 ms
Ours, 2 passes	0.91 cm	0.36°	92%	447 ms
Ours, 3 passes	0.87 cm	0.25°	94%	608 ms
HLoc baseline	0.24 cm	0.18°	100%	878 ms

These numbers come from the *same* `query_eval.py` now reporting ~38 cm on the eval sets. The code did not change between the two; the query set did.

3. The failure on the new eval sets

`python query_eval.py --scene class1 --passes 2` → **median 38.39 cm / 27.1°, 0% recall** over 144 queries. Per-query errors are tightly clustered (~0.35 units / ~27°), which indicates a **systematic** problem, not random matching failure.

4. Diagnosis: the ground-truth poses are wrong

A step-by-step chain, each point independently verifiable:

• **The scene matches.** The query photo (Fig. 1, left) is clearly this room — same desk, door, chair as the training capture.

• **The GT render is blank.** Rendering the splat at the query's *ground-truth* pose (Fig. 1, middle) produces an empty pink wash — the camera sees **no geometry**. A valid in-scene pose cannot render empty.

• **A seed pose renders fine.** The INIT render at MegaLoc's retrieved frame (`frame_00478`, cosine sim 0.42) shows a clean, real view of the same area — so the splat *does* model this region, and the query's true pose is near there.

• **Same coordinate frame.** The query_gt camera centres span x[–1.97, 2.61] y[–0.83, 1.62] z[–6.07, 4.42]; the training frames span x[–1.71, 1.31] y[–1.22, 1.47] z[–7.51, 4.41]. The queries are in the sparse/0 frame — position is not the issue.

• **Position right, orientation wrong.** For q_00013 the nearest training frame is only **0.106 units** away in position, but the query's GT view direction is **56.5°** off it. For q_00100: 0.185 units, but **129.2°** off. These angles differ per query, so it is not a constant convention flip — the orientations are individually noisy.

• **Weak registration.** Each eval query registered against only **43–134** COLMAP 3D points (training frames see far more). Few points → an orientation that is poorly constrained → a GT pose that points off into empty space.



Figure 1. q_00013 panel. Left: real query photo (the room). Middle: splat rendered at the **ground-truth** pose — **blank**. Right: render at the pipeline's final pose. The blank GT render is the key evidence: the registered GT pose does not point at the scene.

5. Why the self-check passes but the queries still fail

This is the subtle point the diagnosis hinges on, and the most likely source of disbelief.

- The COLMAP→nerf **transform** (R, s, t) is fit and self-checked on **training** frames only. It reproduces their centres exactly (resid 0.0000). So the *transform* is correct.
- That transform is then applied **faithfully** to each query's COLMAP pose. If a query's COLMAP pose is itself wrong (noisy orientation from weak registration), a correct transform maps a wrong pose to a wrong pose. Garbage in, garbage out.
- The self-check **cannot** catch this: it never sees the query poses (queries are not in the training set), and a pose's camera *centre* can be nearly right while its *orientation* is badly off — exactly what we measure (centre 0.1 units, direction 56–129°).
- Therefore: pipeline correct + transform correct + **query GT pose wrong** = the splat renders blank at GT, LoFTR cannot match query to an empty render, PnP stays near the bad seed, and the reported error is the error *to a wrong ground truth*.

6. Conclusion

The pipeline is not broken. The new `eval/{class1,stacey}/query_gt.json` poses are unreliable: positions ~right, orientations wrong, caused by weak COLMAP registration (43–134 points/query). We are measuring a correct localiser against incorrect ground truth.

Fixes (data side):

- Re-register the eval queries with a stricter inlier / minimum-point-count threshold.
- Or filter: drop any query whose `num_points3D` is low, or whose GT render has near-zero opacity (blank) — keep only trustworthy GT, then re-measure.
- Sanity: a correct GT pose must render a view that resembles the query photo. Every kept query should pass that eyeball test.

7. How to reproduce / verify every claim

- Pipeline on a good set: `python query_eval.py --scene stacey --passes 2` (expect ~0.9 cm).
- The failure: `python query_eval.py --scene class1 --passes 2` (~38 cm, 0% recall).
- Blank GT render: open `query_eval_out/class1/q_00013_megaloc_summary.png` — middle-top panel is blank (Fig. 1).
- Fit is exact: the run prints `colmap->nerf fit, worst train reproduction error: 0.0000 units`.
- Position vs orientation: nearest-training-frame check gives centre 0.106 u but view-dir 56.5° (q_00013); 0.185 u / 129.2° (q_00100).
- Weak registration: `num_points3D` in `eval/class1/query_gt.json` is 43–134 per query.